

Svarark: Tweets som data

Del 1

1.1 trin 2: hvor stort?

```
print(tweets.shape)
print(list(tweets.columns))

(3945, 7)
['id', 'date', 'content', 'retweets',
 'favorites', 'hashtags', 'mentions']
```

3.945 tweets, 7 oplysninger om hver.

1.1 trin 3: typerne

date er læst som tekst, ikke som dato. Det er ikke en fejl: pandas gætter typen ud fra indholdet, og en dato ligner en tekst.

1.2: én række

```
print(tweets.loc[0, "content"])
print(tweets.loc[100, "content"])
```

Pointen er `.loc[række, kolonne]`: række først, så kolonne.

Del 2: Ét tweet ad gangen

2.1: rens det første

```
print(rens(tweets.loc[0, "content"]))
print(len(rens(tweets.loc[0, "content"])))
```

Output: 22 ord

```
['happy', 'new', 'year', 'from', '#', 'maralago', 'thank',
 'you', 'to', 'my', 'great', 'family', 'for', 'all', 'of',
 'their', 'supportpictwittercom/6usqsiaaj7', '-', 'bei',
 'the', 'mar-a-lago', 'club']
```

Tre ting at få øje på:

- # står helt alene som et ord. Den rå tekst siger # MarALago med et mellemrum. Det er scraperen, der har lagt det ind.
- supportpictwittercom/6usqsiaaj7 er et billedlink, der er klistret sammen med det foregående ord, fordi punktummerne blev fjernet.
- bei er tysk metadata fra Twitters stedangivelse. Der er altså tysk i et engelsksproget datasæt.

Ingen af delene er sproglige fænomener. Det er spor efter, hvordan data blev hentet og det er en anden slags støj end stopord.

Del 3: Tæl ord på tværs

3.1 trin 1 – de ti første

```
antal[o] = antal.get(o, 0) + 1
```

Output: 143 unikke ord i de ti første tweets

3.1 trin 2: alle

```
antal = {}

for t in tweets["content"]:
    for o in rens(t):
        antal[o] = antal.get(o, 0) + 1

print(len(antal))
print(sum(antal.values()))
9698
72555
```

Ca. 9.700 unikke ord ud af ca. 72.500.

3.2: de hyppigste

```
par = []
for nogle, vaerdi in antal.items():
    par.append((vaerdi, nogle))

par.sort(reverse=True)
print(par[:10])

[(2250, 'the'), (2130, '@'), (1918, '#'), (1430, 'to'),
 (1155, 'and'), (1135, 'in'), (1051, 'a'), (1001, 'i'),
 (968, 'of'), (955, 'is')]
```

3.3: til diskussion

- Ingen af de ti siger noget om indholdet. Otte er stopord, og to er tegn.
- @ og # er nummer to og tre på listen. De opstår, fordi scraperen har sat et mellemrum efter tegnet, så `.split()` gør dem til selvstændige ord.
- Stopord er et træk ved sproget, @ og # er et træk ved dataindsamlingen. Den første slags støj er der altid; den anden er en fejl, man kan rette.

Del 4: Hashtags

4.1: tæl dem

```
htal = {}

for h in tweets["hashtags"]:
    if pd.isna(h):
        continue
    for tag in str(h).split(","):
        htal[tag] = htal.get(tag, 0) + 1

print(len(htal))
389 unikke hashtags
1797 hashtags i alt
1186 tweets med mindst ét
[(197, '#Trump2016'), (178, '#MakeAmericaGreatAgain'),
 (91, '#Trump2016pic'), (83, '#MAGA'), (54, '#VoteTrump')]
```

4.2: til diskussion

- Uden `if pd.isna(h): continue` bliver de tomme felter til teksten "nan", fordi `str(NaN)` giver netop det. "nan" ville så stå øverst med 2.759 forekomster – flere end alle rigtige hashtags tilsammen. Det er værd at lade dem prøve.
- 1.797 hashtags fordelt på 1.186 tweets: nogle tweets har flere. Det er derfor feltet skal deles på komma.
- `#Trump2016` og `#Trump2016pic` er samme hashtag, hvor scraperen har hængt `pic` fra et billedlink på. Endnu et spor efter indsamlingen, ikke efter sproget. Med 197 og 91 forekomster er det ikke en detalje.

Del 5: Filtrering med booleans

5.1

```
stor = tweets["retweets"] > 10000

print(len(tweets[stor]))
print(tweets[stor]["favorites"].mean())
print(tweets["favorites"].mean())

baade = stor & tweets["hashtags"].notna()
print(baade.sum())
657
50013.9
18835.4
141
```

657 rækker. Det samme som `stor.sum()`, fordi `True` tæller som 1. De store tweets har cirka 2,7 gange så mange favoritter i gennemsnit som materialet som helhed.

Del 6: To kilder, samme tweets

6.1: overlappet

```
i begge:    3944
kun i A:     1
kun i B:     7
```

6.2: de fire join-typer

```
inner 3944
left  3945
right 3951
outer 3952
```

Det er hele pointen med opgaven: forskellen mellem de fire er 1, 7 og 8 rækker ud af næsten 4.000. Ingen ser det ved at kigge. De skal tælle.

6.3: sammenlign kilderne

- Punkt 1 er et trickspørgsmål. content findes kun i kilde A, så den kan slet ikke sammenlignes. De kolonner, der findes begge steder, er date, retweets og favorites, og efter koblingen hedder de _a og _b.

```
print((samlet["retweets_a"] == samlet["retweets_b"]).mean())
print((samlet["favorites_a"] == samlet["favorites_b"]).mean())
print((samlet["date_a"] == samlet["date_b"]).mean())
print(samlet["geo"].notna().sum())

0.0
0.0
0.0
0
```

Ikke ét eneste tweet har samme retweet-tal, samme favorittal eller samme tidsstempel i de to kilder. Medianforskellen på retweets er 94. geo er tom for samtlige 3.944 rækker.

6.4: til diskussion

- Ingen af kilderne er forkert. Et retweet-tal er ikke en egenskab ved en tweet, men en måling foretaget på et tidspunkt. To scrapes på to tidspunkter giver to tal. Det, der skal med i en metodesektion, er download-datoen.
- Tidsforskellen er 7 timer for 3.665 af rækkerne og 6 timer for 279. Den klumper omkring sommertidsskiftene, så de to kilder håndterer tidszoner forskelligt. Man behøver ikke kunne forklare præcis hvordan, pointen er bare, at et tidsstempel ikke er en neutral oplysning.
- En tom kolonne skal ikke bare slettes. At geo er tom for alle 3.944 tweets er selv en oplysning. Enten var stedangivelse slået fra, eller også hentede scraperen den ikke. Det er forskellen på "ingen data" og "vi ved det ikke".

6.5: tjek_kobling

```
def tjek_kobling(venstre, hoejre, noegle):
    ny = pd.merge(venstre, hoejre, on=noegle)
    print("Rækker venstre:", len(venstre))
    print("Rækker højre: ", len(hoejre))
    print("Rækker efter: ", len(ny))
    kun_v = set(venstre[noegle]) - set(hoejre[noegle])
    kun_h = set(hoejre[noegle]) - set(venstre[noegle])
    print("Kun i venstre:", len(kun_v))
    print("Kun i højre: ", len(kun_h))
```

Med 3.945 nøgler skal funktionen printe antallet, ikke selve nøglerne. Det er en lille, men vigtig ændring fra lektion 3, hvor der kun var tre partier.

Del 7: Måneder og form

7.1: træk måneden ud

`pd.to_datetime()` laver teksten om til en rigtig dato, og `.dt.month` trækker måneden ud. Uden det første skridt virker det andet ikke.

```
1: 477    2: 472    3: 420    4: 270    5: 342    6: 291
7: 353    8: 272    9: 268   10: 469   11: 180   12: 131
```

7.2: koblingen med opslagstabellen

```
inner 3814
left  3945
```

131 tweets forsvinder, hele december. Opslagstabellen har kun 11 måneder. Tabellen ser komplet ud, koblingen giver ingen fejl, og et helt månedsudsnit af materialet er væk. Og det er ikke et tilfældigt udsnit.

7.3 trin 1: januar

```
477 tweets i januar
"great" optræder 72 gange
```

7.3 trin 2 til 4: langt og bredt

```
ord_valg = ["great", "thank", "america"]
raekker = []

for m in sorted(tweets["maaned"].unique()):
    dm = tweets[tweets["maaned"] == m]
    tael = {}
    for t in dm["content"]:
        for o in rens(t):
            tael[o] = tael.get(o, 0) + 1
    for w in ord_valg:
        raekker.append({"maaned": m, "ord": w,
                        "antal": tael.get(w, 0)})

lang = pd.DataFrame(raekker)
bred = lang.pivot(index="ord", columns="maaned", values="antal")

maaned    1    2    3    4    5    6    7    8    9   10   11   12
ord
america   21   40   25   21   24   19   19   11   20   25   14    4
great     72   65   52   59   55   40   47   32   43   42   40   21
thank     73   65   75   65   53   41   41   47   67   64   26   16
```

Langt format: 36 rækker (12 × 3). Bredt: 3 rækker og 12 kolonner.

Del 8: Funktionen

```
def tael_ord(tekster):  
    antal = {}  
    for t in tekster:  
        for o in rens(t):  
            antal[o] = antal.get(o, 0) + 1  
    return antal
```

Den sparer fire linjer hver gang, den bruges og i 7.3 blev den brugt tolv gange, én gang per måned. Løkken i trin 2 skrumper fra ni til fem linjer. Den vigtigere pointe: skal rens() rettes (og det skal den garenteret på et tidspunkt), skal rettelsen kun laves ét sted.